COSC2673 Computational Machine Learning - Assessment 2

Student: Alissa Nguyen (s4085541)

0. Introduction

This report describes the machine learning pipeline for predicting wildfire intensity levels using the provided wildfire dataset. This task requires the prediction of the wildfire intensity class based on environmental, geographical, temporal, weather, and satellite derived features. As wildfires represent an extremely dangerous natural disaster, their intensity predictions could affect the planning of emergency response resources. Falsely predicted fire intensity, even by just one class level higher than the real one, could impact the preparation of response teams and the wasting of valuable resources. Hence, the aim of this assignment is not only to identify the optimal model, a model with highest validation score, but also to examine its characteristics and reliability of the prediction. This report analyses and compares three models: Decision Tree, Support Vector Machine (SVM), and Neural Network. The task requires the implementation of these three models based on justification and data analysis. Model development includes such steps as data analysis, hyperparameter tuning experiments, model evaluation, performance comparison and discussion, final model selection and generating predictions for the provided test dataset. The final model was chosen based on validation macro F1-score, considering class imbalance in the target variable. Additionally, the final decision was influenced by accuracy, weighted F1-score, class level F1-scores, confusion matrices, and training vs validation scores gaps.

Before running the model, ensure that:

1. A Python environment is installed (Anaconda or standard Python with Jupyter Notebook)
2. Place the following files in the same folder as the notebook: wildfire_cls_train_full.csv and wildfire_cls_test_features.csv
3. Make sure you have the following libraries installed: pandas, numpy, matplotlib, scikit-learn

To run this notebook

4. Start jupyter notebook/lab
5. Run the cells from top to bottom order
6. The notebook will load the data, inspect and analyse the dataset, preprocess the features, train and evaluate three machine learning models, compare their performance, and generate the final prediction CSV.
7. The final prediction file will be saved as s4085541_predictions.csv (this file must contain one column only fire_intensity and predictions must remain in the same order as the rows in the provided test dataset).

A. Task Definition and Dataset Description

Target variable fire_intensity represents the intensity of wildfire events.
The four target classes are

Encoded value	Fire intensity class
0	Low
1	Moderate
2	High
3	Extreme

Table 1. Fire intensity class labels

The training set wildfire_cls_train_full.csv contains 4340 records and 20 columns. The test set wildfire_cls_test_features.csv includes 1085 records and 19 columns. The test dataset does not include the target column. Hence it was not used during training and validation stage. The test set was only used for making predictions after selecting the final model. The notebook checks whether the test set features columns match the training set features columns once fire_intensity target was removed from the training set.

Dataset	Rows	Columns	Target included?	Purpose
wildfire_cls_train_full.csv	4340	20	Yes	EDA, preprocessing, model training, validation, model selection
wildfire_cls_test_features.csv	1085	19	No	Final prediction

Table 2. Dataset overview

In addition to the target fire_intensity, the dataset contains other types of information. All features were grouped into categories:
Geographic features: latitude, longitude.
Temporal features: acq_date, acq_time, year, month, season, daynight.
Location descriptors: region, country.
Fire and detection descriptors: fire_type, satellite, instrument, brightness_k, confidence.

Weather-related features: temp_max_c, wind_max_kmh, precip_mm, humidity_pct.
The main challenges in this dataset are missing values, mixed feature types, class imbalance, and class overlap. There are missing values appear in month, brightness_k, and wind_max_kmh. Additionally, there are several categorical features which need to be encoded for modelling. The target variable classes distribution is unequal with more examples of Moderate and High fires and less number of Extreme cases.

B. Exploratory Data Analysis and Data Handling

An exploratory data analysis was conducted to better understand the dataset before modelling. The focus of this analysis was on target distribution, missing values, numerical and categorical feature distributions, feature target relationships, and correlations between numerical variables. This is useful because the dataset contains several different feature types, and the models especially require clean numerical input. Only the labelled train dataset, 'wildfire_cls_train_full.csv', is used for exploratory data analysis since this dataset comprises the target feature, 'fire_intensity'. However, the test dataset provided does not have any label, hence only used for prediction purposes.

B.1. Target class distribution

fire_intensity	count	percentage
Moderate	1921	44.26
High	1340	30.88
Low	698	16.08
Extreme	381	8.78

Table 3. Distribution of fire intensity classes

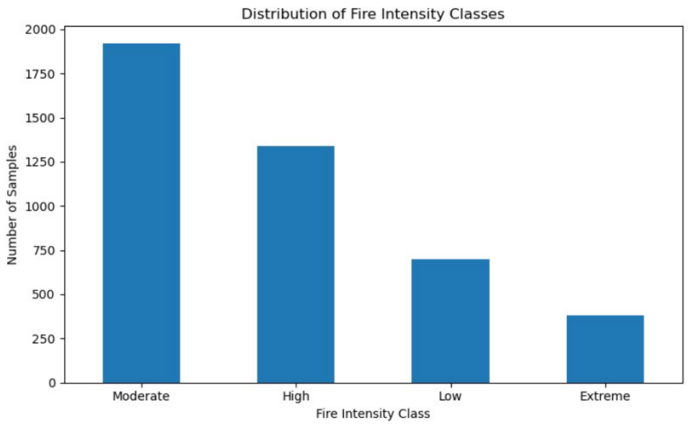

Figure 1. Distribution of fire intensity classes

Figure 1 demonstrates the distribution of the target variable, fire_intensity. The largest class is Moderate, followed by High, Low, and Extreme. The Extreme class has the smallest number of samples (less than 10%) of all the four fire intensity levels. That means that the classification is imbalanced.

It was necessary to produce this figure since class distribution directly impacts model evaluation and performance. A classifier could be trained to accurately classify all classes except the smallest one. Therefore, accuracy metric cannot be the sole way to measure a model's performance, even if it is reasonably high. On this particular task, the smallest Extreme class is important since it corresponds to the highest risk fires. Therefore, macro F1-score was added as an additional model evaluation metric along with accuracy and weighted F1-score. Since macro F1-score assigns equal weight to all classes, it is the right way to evaluate imbalanced tasks with multiple classes.

B.2 Missing value analysis

```
Missing values in training data:
month          416
brightness_k   327
wind_max_kmh   207
dtype: int64
```

Table 4. Missing value summary

EDA showed that the training dataset has missing values in month, brightness_k, and wind_max_kmh, while the test dataset has no missing values. It was important to investigate missing values beforehand since all the models that will be used for classification cannot be trained on missing data.

Rows containing missing values cannot be dropped from the dataset since that would reduce the number of observations available for training. Moreover, missing values are in meaningful features. Hence, they were handled through median imputation for numerical values and most-frequent-category imputation for categorical features. The first strategy is chosen since numerical values corresponding to weather and satellite observation can vary greatly.

Handling missing values in this manner helps with reproducibility of the analysis and experiments. Imputation as part of the preprocessing workflow ensures consistency of transformation on the training, validation, and final test prediction datasets.

B.3 Numerical feature distributions

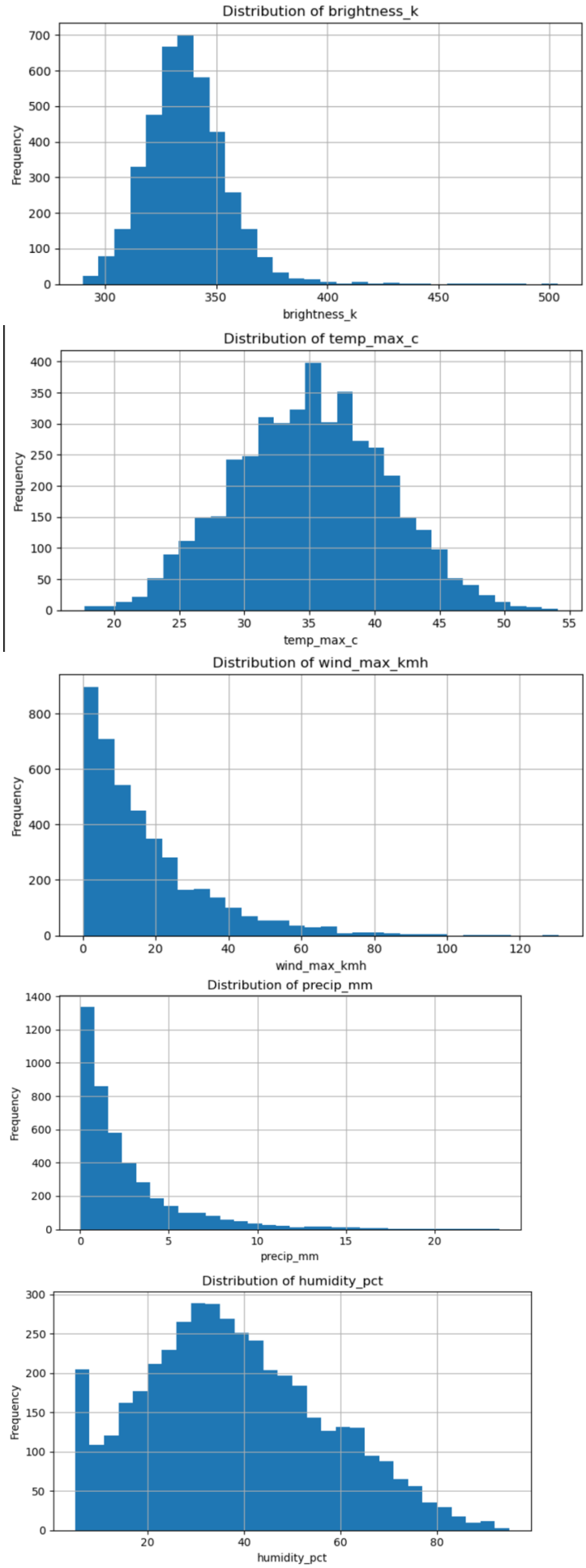

Figure 2. Numerical feature distributions

Figure 2 shows that some of the features are on significantly different scales. Latitude and longitude span large geographic ranges. Weather-related features such as temp_max_c, wind_max_kmh, precip_mm, and humidity_pct have their distinct units and scales. brightness_k has different unit (Kelvin) and is related to satellite fire observation.

Several patterns are noticeable. Brightness_k shows right skewness, i.e., most of the observations are located within the lower-to-middle range, and there are few bright observations. Similarly, wind_max_kmh and precip_mm have right skewness. Maximum temperature has closer to normal distribution, while humidity shows broader distribution. EDA showed that numerical features are on different scales and supports scaling for SVM and NN models.

This exploration is important since some models are more sensitive to scale and variance than others. Therefore, numerical scaling was included in the preprocessing workflow. Although Decision Tree models do not care about scale that much, using scaling ensured that the models will be compared fairly without additional complexity of handling different scales.

B.4 Categorical feature distributions

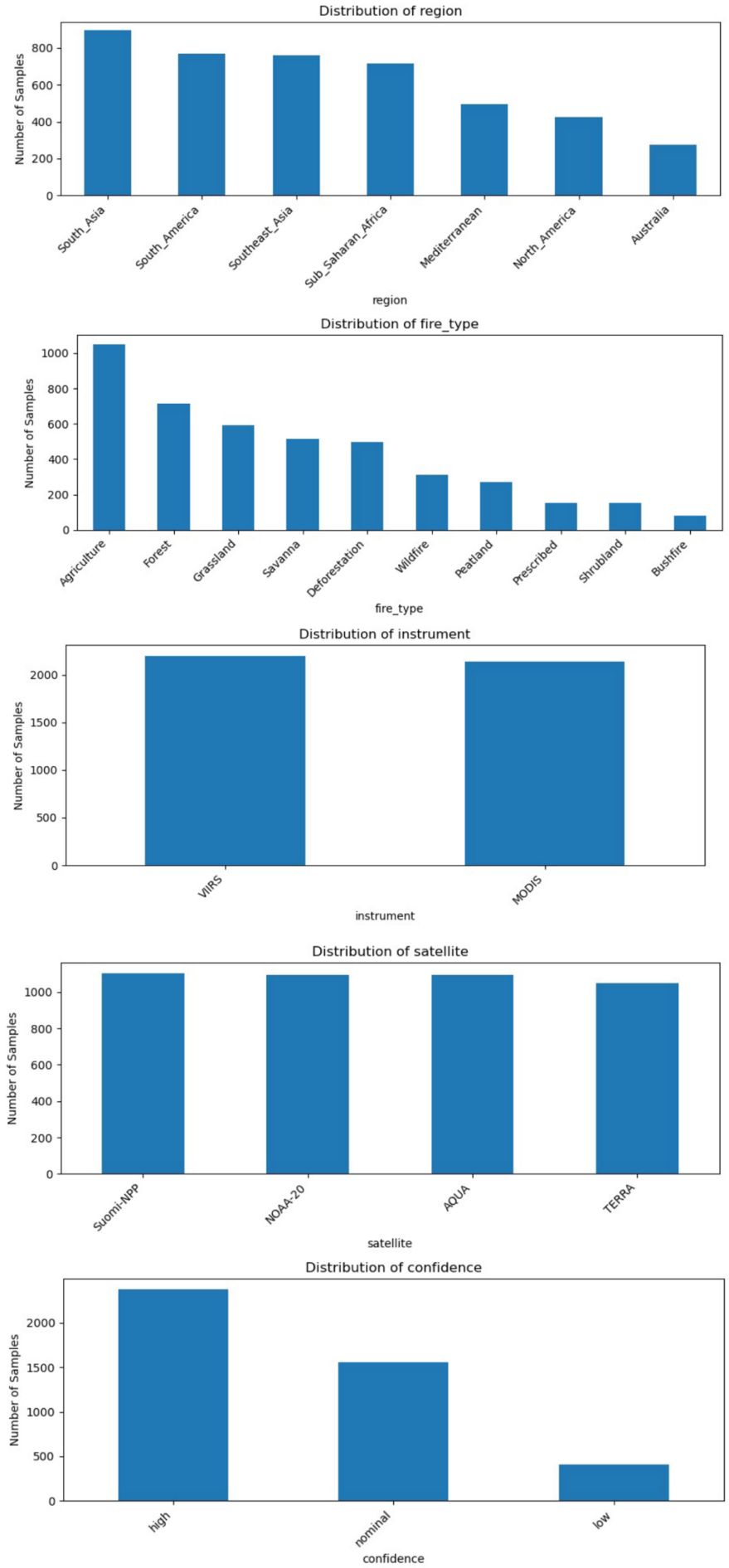

Figure 3. Categorical feature distributions

B.5 Numerical feature relationships with fire_intensity

Figure 3 shows the distribution of the categorical variables. Several categorical variables are present in the dataset, such as season, daynight, region, country, fire_type, satellite, instrument, and confidence. Some variables have a small cardinality (daynight, instrument), while others have high cardinality (country, acq_date).

According to the notebook, acq_date variable has 709 unique dates, country has 35 countries, fire_type has 10 categories, region has 7, and the rest have fewer categories. Having many categories poses a challenge since one-hot encoding creates as many new columns as categories.

This plot indicates that categorical features can contain useful information for classification. For instance, fire intensity distribution among different fire types and regions might not be equal. However, categorical features have to be transformed before being fed to the classifiers. Thus, all categorical variables were one-hot encoded in preprocessing.

This plot shows that categorical information is useful but not immediately obvious since the data is not distributed uniformly. One-hot encoding was used to incorporate categorical information and not treat acq_date as individual dates.

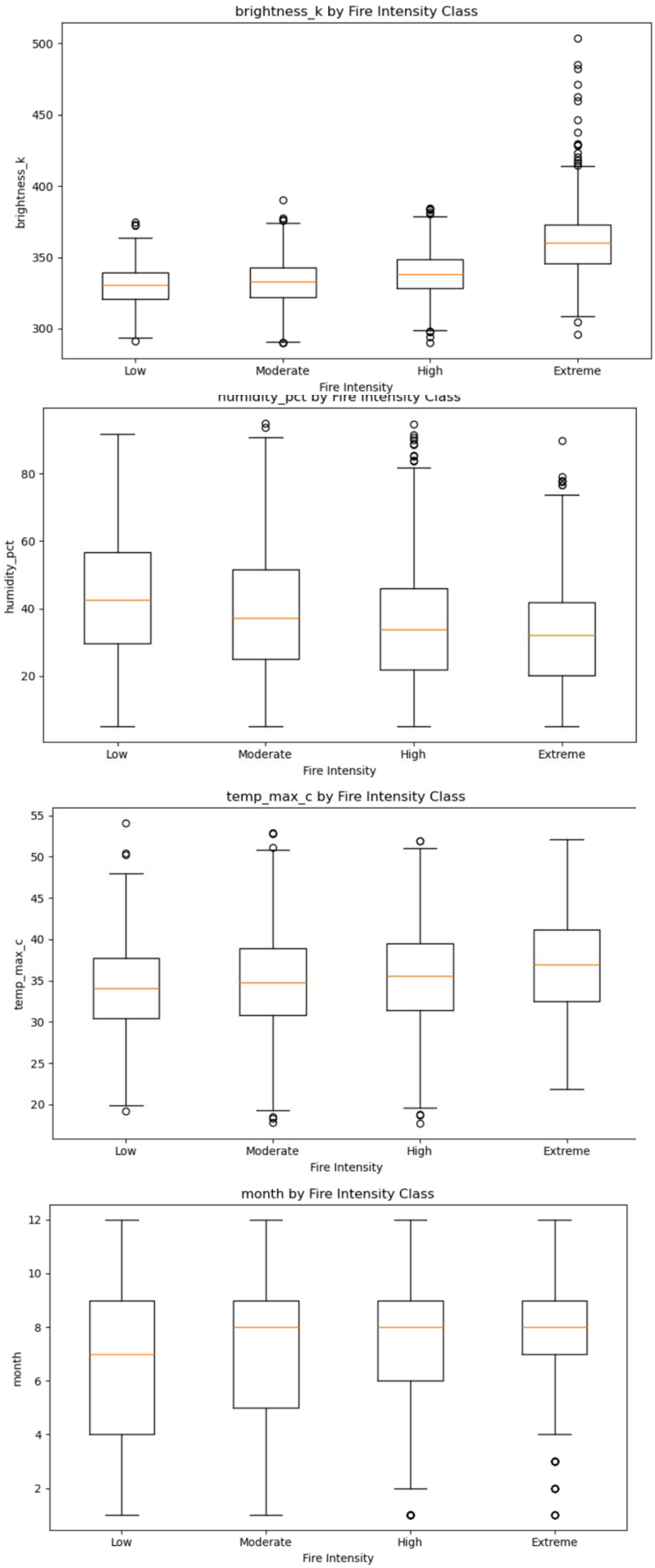

Figure 4. Numerical feature boxplots by fire intensity class

Figure 4 plots the distributions of key numerical features per fire intensity level. This plot is useful for determining whether the value of the feature changes substantially for Low, Moderate, High, and Extreme intensity fires. Feature with different distribution across classes can help classifiers differentiate between classes.

The clear pattern in the plot is brightness_k. Brightness_k shows the largest median and spread in Extreme fires compared to the other three classes. It is not surprising because brightness temperature depends on the fire activity observed by satellites. Temperature maxima are slightly increasing as we progress through higher intensity fires. Humidity percent shows decreasing median among higher intensity fires, which also is expected behaviour. Wind maxima have somewhat lower values at Extreme fire intensity.

This plot is important because it provides the first indication that some numerical features have relationship with fire intensity levels. Also, it shows that the classes are not strictly separable. If the classes were fully separable by a feature, then the classifier would not have performed moderately well on validation data.

B.6 Categorical feature relationships with fire_intensity

EDA explored relationships between fire intensity levels and categorical features by calculating

Class proportions by season:

fire_intensity season	Low	Moderate	High	Extreme
Autumn	0.183	0.448	0.274	0.095
Spring	0.191	0.458	0.284	0.067
Summer	0.115	0.414	0.360	0.110
Winter	0.164	0.456	0.301	0.080

Class proportions by daynight:

fire_intensity daynight	Low	Moderate	High	Extreme
D	0.163	0.446	0.30	0.091
N	0.158	0.438	0.32	0.084

Class proportions by region:

fire_intensity region	Low	Moderate	High	Extreme
Australia	0.076	0.393	0.400	0.131
Mediterranean	0.082	0.433	0.368	0.117
North_America	0.087	0.333	0.397	0.183
South_America	0.069	0.411	0.392	0.128
South_Asia	0.296	0.489	0.197	0.018
Southeast_Asia	0.225	0.477	0.240	0.058
Sub_Saharan_Africa	0.154	0.473	0.303	0.070

Table 5. Selected categorical class proportions

proportional distribution within the categories. This analysis was important since categorical information cannot be shown in a regular correlation heatmap. Class proportionality table was produced for season, daynight, region, country, fire_type, satellite, instrument, and confidence.

Some clear patterns were noticed. In particular, North America had a relatively high portion of Extreme fires, while South Asia and Southeast Asia had higher amounts of Low and Moderate fires. Fire_type Wildfires and Bushfires had higher portions of Extreme fires, while Grasslands and Peatlands had smaller percentages of the latter.

This analysis is helpful since it provides the evidence that categorical features have predictive power regarding fire intensity classes. For this reason, these variables were kept in the modelling and one-hot encoded.

B.7 Correlation analysis between numerical features

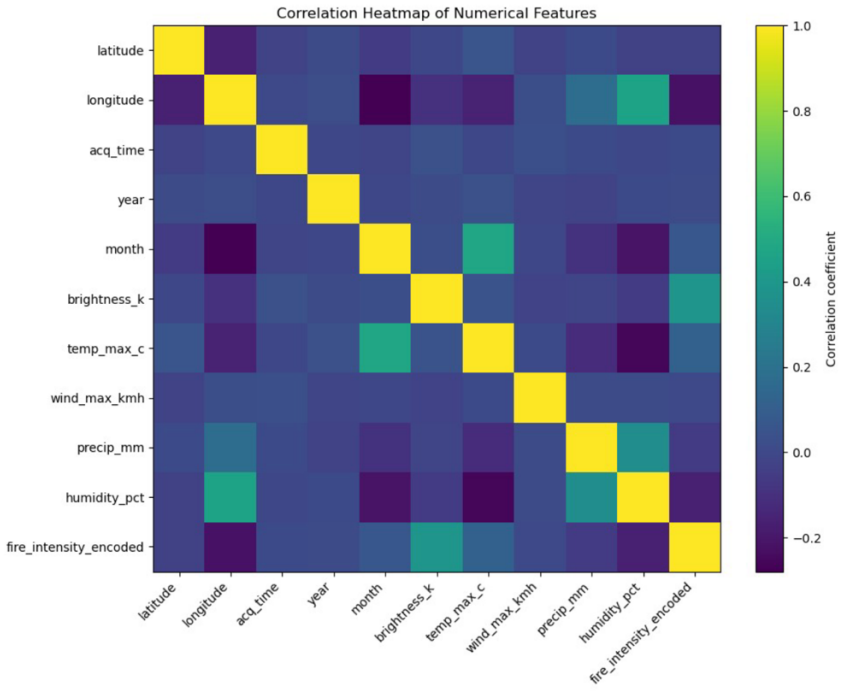

Figure 6. Correlation heatmap of numerical features

Correlation heatmap allows seeing relationships between all numerical features of the dataset. This exploration allows detecting possible correlations between numerical features.

The relationships were identified for several features. Maximum temperature and month had moderate positive correlation coefficient of approximately 0.476. On the contrary, humidity percent and maximum temperature had negative correlation (approximately -0.163). humidity percent and longitude have a positive correlation coefficient (approximately 0.462).

This EDA is necessary because it determines correlations between numerical features and allows understanding how it might influence the model. However, it does not necessarily imply that some features should be removed, since some models allow dealing with multiple features.

B.8 Correlation between numerical features and target

One of the most important EDA outputs is the target-correlation rank plot. This figure ranks numerical features according to how strong is their linear correlation with the target.

The strongest correlation was found for brightness_k with coefficient of 0.389. Other top-

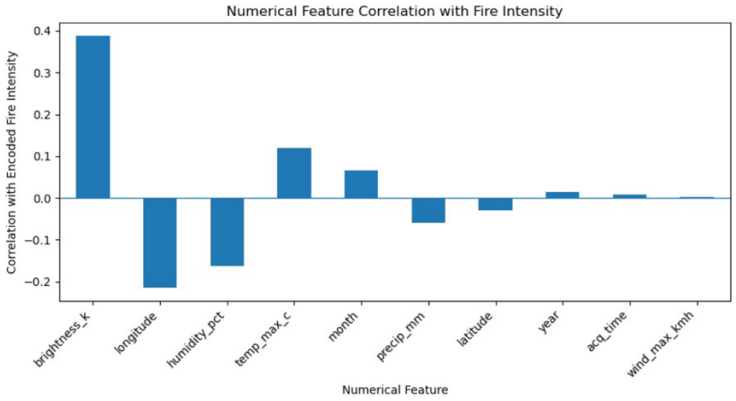

Figure 7. Ranked numerical feature correlation with fire_intensity

ranked features were longitude (-0.215), humidity_percent (-0.163), and temp_max_c (0.119).

This figure shows that brightness temperature has the strongest correlation with fire intensity classes. It is reasonable since the assignment states that brightness_k is related to fire activity. Negative correlation with humidity percent shows that higher humidity tends to correlate with lower fire intensity. Finally, negative correlation with longitude has unclear physical meaning and likely indicates regional distribution of samples.

Target-correlation was a powerful tool that allowed ranking numerical features. However, only one of the features with the strongest correlation was included in further experiments. The reason is that linear correlation does not capture all kinds of feature-target relationships.

EDA has directly influenced preprocessing decisions. Missing values require imputation, while presence of categorical variables means one-hot encoding. Presence of features with vastly different scales requires scaling for SVM and NN models. Finally, imbalanced classes lead to selection of macro F1-score as the main evaluation metric.
The final preprocessing pipeline consisted of the following steps:

1. Separate input features from the target.
2. Encode fire intensity variable into numeric target.
3. Process acq_date and acq_time.
4. Fill missing values (numerical values with median, categorical with the most frequent).
5. Encode categorical features with one-hot encoding.
6. Scale numerical features using standard scaler.
7. Stratify labelled data into training and validation.

A pipeline-based preprocessing workflow has been developed. However, it does not imply adding another model. It only provides a convenient way of combining several preprocessing steps into a single object.

C. Model Development and Performance Analysis

Model Development and Performance Analysis Three models were trained and evaluated as part of this assignment, namely Decision Tree, SVM, and Neural Network. The choice of models was determined by the assessment requirements since only those three types had to be tested. These models represent different approaches to classification and testing their relative performance. While the Decision Tree model aims at providing rule-based classification through a sequence of logical decisions, SVM tests for separation between classes with margin, and the Neural Network model tests a more complex and non-linear solution. All of these models were applied to the wildfire_cls_train_full.csv and tested against validation macro F1-score. The actual test data in the wildfire_cls_test_features.csv file was not used in the experiments since it did not include fire_intensity labels. In addition to the selected evaluation metric, accuracy and weighted/individual F1-score were also calculated for each model to supplement performance evaluation.

C.1 Decision Tree

Decision Tree Decision Tree model was used first because it is interpretable and flexible enough to account for some simple non-linear patterns. This model is a good starting point for solving this classification problem related to wildfire intensity. According to the Week 5 lectures, tree-based models split the predictor space into smaller segments and build tree-like structures that can be used to make predictions. This approach allows separating different classes through feature thresholds. Therefore, Decision Tree is applicable to this dataset because some wildfire features may exhibit threshold-type relationships with fire intensity. The advantage of this model is clear from its name Decision Trees are easy to visualise and analyse. They provide insights into features used by the classifier and allow interpreting the results in terms of feature importance.

The disadvantage of this model is that Decision Trees require careful tuning to avoid underfitting or overfitting. On one hand, a shallow tree will not provide enough splitting criteria and will underfit data, and, on the other hand, a deeply nested tree will memorise the training dataset and overfit. In addition, according to the Week 5 lecture materials, tree-

based models are simple and useful, yet they may not be the most efficient models because of their simplicity and interpretability.

C1.1 Hyperparameter experiment

max_depth To assess how tree depth impacts training and validation accuracy, several tree depths were used in this experiment.

	model	train_accuracy	val_accuracy	train_macro_f1	val_macro_f1	train_weighted_f1	val_weighted_f1	max_depth
1	Decision Tree max_depth=3	0.479263	0.461982	0.373706	0.356689	0.437003	0.420562	3.0
4	Decision Tree max_depth=6	0.533986	0.471198	0.419885	0.337223	0.471263	0.402996	6.0
5	Decision Tree max_depth=8	0.579781	0.444700	0.511845	0.334788	0.543748	0.397082	8.0
7	Decision Tree max_depth=12	0.710829	0.407834	0.690862	0.333657	0.700648	0.386904	12.0
3	Decision Tree max_depth=5	0.511521	0.478111	0.372572	0.331445	0.435328	0.399483	5.0
8	Decision Tree max_depth=None	1.000000	0.375576	1.000000	0.325011	1.000000	0.372147	NaN
2	Decision Tree max_depth=4	0.492512	0.471198	0.343135	0.322792	0.402914	0.377788	4.0
6	Decision Tree max_depth=10	0.644873	0.408986	0.592976	0.296764	0.615584	0.356416	10.0
0	Decision Tree max_depth=2	0.469470	0.480415	0.276657	0.292683	0.350289	0.359153	2.0

Table C1.1 Decision Tree max_depth results

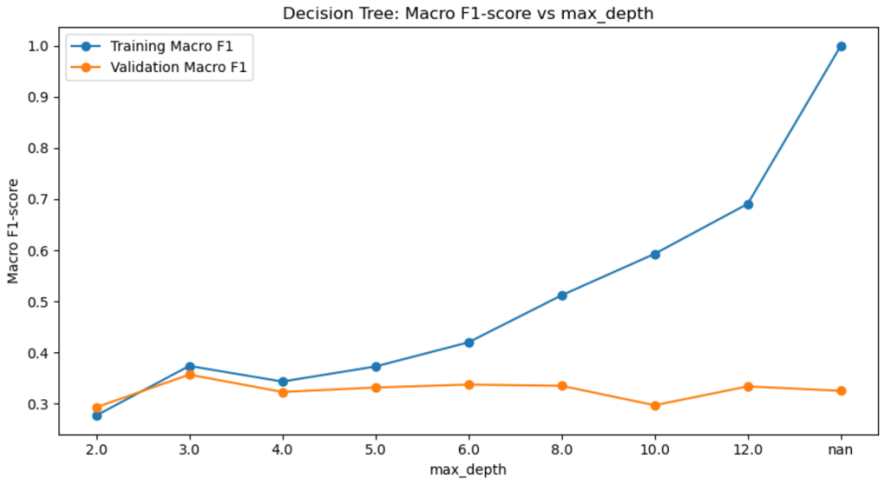

Figure C1.1 Decision Tree macro F1-score vs max_depth

From the performance curve, it is possible to see that as max_depth grows, training accuracy and macro F1-score grow too. However, this is not necessarily true for validation performance: if training score keeps rising while validation score stays flat or decreases, the model overfits. This way, it is easy to identify the range of values when the model is stable and reliable and when overfitting starts.

The best Decision Tree in my experiment used:

- Best setting: max_depth = 3
- Validation accuracy: 0.462
- Validation macro F1: 0.357

It can be concluded that the model selected based on this experiment had the worst validation macro F1-score, but the lowest training-validation gap. The gap was roughly equal to 0.017, meaning that this model was relatively robust and not prone to overfitting. At the same time, low validation macro F1-score suggests that the model failed to separate all classes with the highest efficiency possible.

Overall, the Decision Tree was useful as an interpretable baseline. It was stable and easy to explain, but its lower validation macro F1-score suggests that its simple decision rules were not enough to separate the wildfire intensity classes as well as the more flexible models.

C.2 Support Vector Machine

Support Vector Machine SVM (support vector machine) model was used as another strong model, but with different assumptions. In the Week 6 lectures, SVMs are explained as models aiming at finding decision hyperplane between classes. In case of noisy data, SVM can still find the boundary, albeit at the expense of misclassifying some training examples. The RBF kernel expands the SVM to a more complicated structure where a non-linear boundary can be found. SVM is a suitable model for this dataset because fire intensity does not seem to be separated based on one feature alone, but rather on feature combinations. Wildfires' intensity is associated with satellites' measures of brightness in certain

areas at specific moments in time, weather conditions, locations and fire types. Linear SVM can check whether fire intensity classes are separated by a linear boundary, while the RBF kernel model can check for non-linear boundaries. One of the advantages of SVMs is that they can learn complicated structures. On the downside, they are less interpretable than tree-based models, and they can be computationally heavy when combined with one-hot encoding of features. They are also sensitive to scaling, which is why standardisation of numerical predictors was included in the preprocessing step.

C.2.1 Hyperparameter experiment: kernel and C

- linear: a simpler linear decision boundary
- rbf: a more flexible non-linear decision boundary

Several C values were explored too: 0.1, 1, and 10. The regularisation parameter C indicates the model's preference to classify samples correctly. Lower C leads to more relaxed training because the model can afford to misclassify some examples. Higher C forces the model to separate the samples more accurately, which results in overfitting in extreme cases.

	model	train_accuracy	val_accuracy	train_macro_f1	val_macro_f1	train_weighted_f1	val_weighted_f1	kernel	C
5	SVM kernel=rbf, C=10	0.948445	0.434332	0.948504	0.385530	0.948329	0.425663	rbf	10.0
4	SVM kernel=rbf, C=1	0.572005	0.480415	0.436058	0.329504	0.506798	0.402078	rbf	1.0
0	SVM kernel=linear, C=0.1	0.476094	0.470046	0.312534	0.317769	0.381015	0.380515	linear	0.1
1	SVM kernel=linear, C=1	0.474942	0.452765	0.328750	0.317020	0.394149	0.374613	linear	1.0
2	SVM kernel=linear, C=10	0.477247	0.451613	0.336104	0.314885	0.397120	0.373548	linear	10.0
3	SVM kernel=rbf, C=0.1	0.445565	0.444700	0.158564	0.159270	0.278318	0.278926	rbf	0.1

Table C2.1 SVM hyperparameter results

From the results of the hyperparameter experiment, it becomes evident that the RBF kernel model performs better than the linear one. The RBF kernel with C = 10 achieved the highest validation macro F1-score among all three model tested. This suggests that fire intensity classes in the provided dataset are not separable with a simple linear boundary. The RBF boundary appears to be more appropriate for this task and resulted in the highest score. However, there are also obvious signs that the SVM model may be overfitting. Training macro F1-score was approximately 0.949, while validation macro F1-score was approximately 0.386. In other words, the difference between training and validation macro F1-score is quite big, indicating overfitting.

C2.3 Best SVM Evaluation

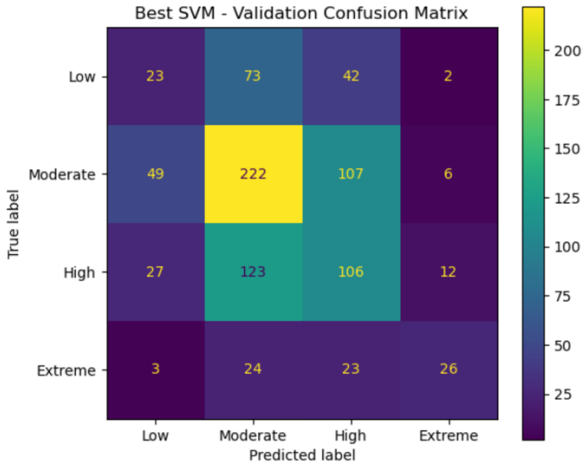

Figure C2.3 Best SVM confusion matrix

The best SVM model was overfitting the training set, but there were some interesting results regarding class distribution. Most notably, some Low samples were misclassified as High and some vice versa. This can be caused by the fact that fire intensity classes overlap, therefore the classifier is unable to distinguish between some classes, no matter the complexity of the boundary learned. As it has already been mentioned, validation accuracy does not provide much information regarding this task since it does not show which classes are more often misclassified. From a real-world perspective, over-classification of Extreme fires with Moderate would be worse than some errors between Moderate and Low. Therefore, the confusion matrix and macro F1-score are essential for analysing this classifier. Overall, SVM was the best classifier among three options tested here, but with a significant overfitting issue.

C.3 Neural Network

Multi-Layer Perceptron was used as the neural network model because of the availability of preconfigured implementation in the scikit-learn library. This classifier was not implemented with PyTorch because the aim of this assignment was to explore the required classifiers clearly within the COSC2673 scope. The neural network classifier was used as an alternative to the SVM due to its ability to learn complex patterns in data. In the case of the current wildfire dataset, it is expected that there is a nonlinear relationship between some input features and target class. Moreover, there are many inter-dependent features, therefore the MLP seems like a suitable choice.

C3.1 Hyperparameter experiment: learning rate

For this model, I decided to focus on the impact of the learning rate and chose to explore several different values. In particular, I examined the following: 0.0001, 0.001, 0.01, 0.1

The number and width of hidden layers remained the same across all runs of the experiment so that the experiment focused on the effect of the learning rate.

	model	train_accuracy	val_accuracy	train_macro_f1	val_macro_f1	train_weighted_f1	val_weighted_f1	learning_rate
1	Neural Network learning_rate=0.001	0.994816	0.415899	0.994850	0.377674	0.994815	0.412831	0.0010
0	Neural Network learning_rate=0.0001	0.607431	0.442396	0.548033	0.376402	0.588785	0.415975	0.0001
2	Neural Network learning_rate=0.01	0.976382	0.406682	0.975351	0.365190	0.976391	0.404505	0.0100
3	Neural Network learning_rate=0.1	0.548099	0.457373	0.475408	0.358097	0.517244	0.421798	0.1000

Table C3.1 Neural Network learning rate results

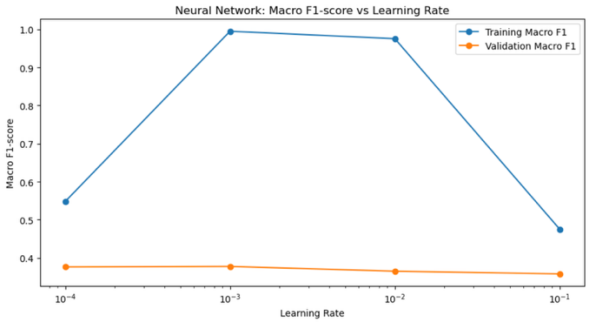

Figure C3.1 Neural Network macro F1-score vs learning rate

Even though learning_rate = 0.1 provided the maximum accuracy during validation, the model was still not considered to be the best Neural Network because of the lower validation macro F1-score. Therefore, it demonstrates why the measure of accuracy cannot be used solely to evaluate models when it comes to such an imbalance of the target classes. Based on the learning rate, we can conclude that the model depends on this hyperparameter. The training with the lowest accuracy occurred at the smallest learning rate, which means that the latter was too low for the neural network to train properly in terms of the maximum number of iterations. At the same time, with learning_rate = 0.001, the best validation macro F1-score was observed, but there was a significant difference between training and validation performances, which suggests overfitting of the neural network.

C3.2 Loss curve analysis

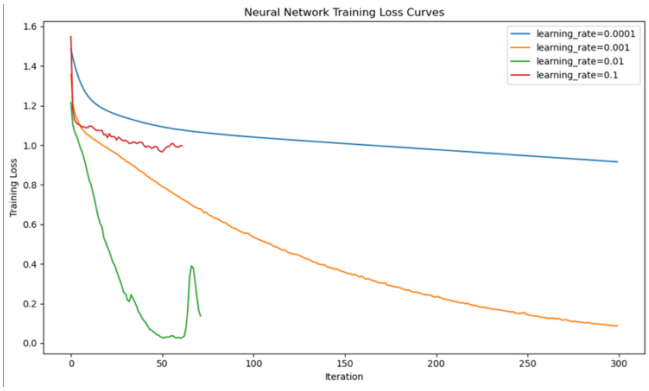

Figure 3.2 Neural Network training loss curves

Loss curves were significant since they represented how the model trained. A stable training process should normally have a decrease in loss during the process. For learning_rate = 0.001, there was consistent decrease in loss indicating stability. With learning_rate = 0.0001, the loss decreased very slowly, indicating slow learning. With learning_rate = 0.01, there was quick decrease in loss but instability in the latter stages. Learning_rate = 0.1 was unstable and could not get the maximum F1-score. The Best NN had almost the same validation macro F1-score as the SVM but did not outperform it. There is also training-validation gap for the best NN. Training macro F1-score was approximately 0.995, while validation macro F1-score was approximately 0.378.

C3.3 Best Neural Network evaluation

Best Neural Network – Validation Classification Report

	precision	recall	f1-score	support
Low	0.2481	0.2357	0.2418	140
Moderate	0.5037	0.5365	0.5195	384
High	0.3750	0.3358	0.3543	268
Extreme	0.3721	0.4211	0.3951	76
accuracy			0.4159	868
macro avg	0.3747	0.3823	0.3777	868
weighted avg	0.4112	0.4159	0.4128	868

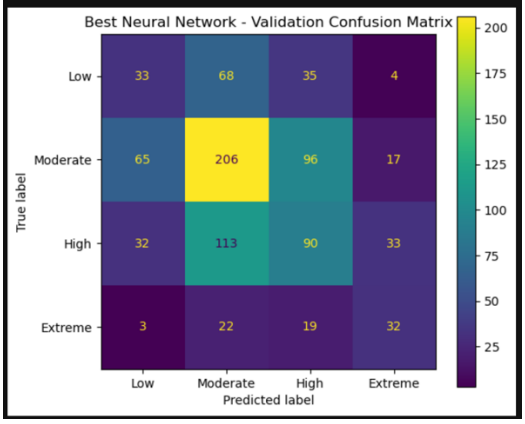

Figure C3.3 Best Neural Network confusion table and matrix notebook screenshot

The Neural Network confusion matrix indicated that the prediction of the Moderate category performed better compared to others, yet it misclassified most instances of the Low and High categories as Moderate. The Extreme category had some correct predictions; however, the lower number of supports implies that its performance needs further clarification. In general, the Neural Network algorithm had great flexibility, allowing for learning of complicated patterns; however, this flexibility did not ensure the highest performance during validation. This algorithm was sensitive to the learning rate and had a large difference between training and validation; therefore, it was less preferred to the SVM algorithm.

C4. Summary of development findings

model_family	model	train_accuracy	val_accuracy	train_macro_f1	val_macro_f1	train_weighted_f1	val_weighted_f1	accuracy_gap	macro_f1_gap
SVM	SVM kernel=rbf, C=10	0.948445	0.434332	0.948504	0.385530	0.948329	0.425663	0.514113	0.562974
Neural Network	Neural Network learning_rate=0.001	0.994816	0.415899	0.994850	0.377674	0.994815	0.412831	0.578917	0.617176
Decision Tree	Decision Tree max_depth=3	0.479263	0.461982	0.373706	0.356689	0.437003	0.420562	0.017281	0.017017

C4. Summary of best model results

The model development results show that each model has different strengths and weaknesses. The Decision Tree was the easiest to interpret and had the smallest training-validation gap, but it had the weakest validation macro F1-score. The SVM achieved the highest validation macro F1-score, suggesting it was the strongest model under the main evaluation metric. The Neural Network performed close to the SVM but had a larger overfitting gap and was more sensitive to the learning rate. The results also show that more complex models do not automatically generalise better. Both the SVM and Neural Network achieved very high training performance, but their validation performance was much lower. This suggests that flexibility helped them fit the training data but also increased overfitting risk. This trade-off is important for final model selection.

D. Model Comparison

Model comparison was made based on the same validation data, which were derived from the file wildfire_cls_train_full.csv. It was necessary to provide a fair comparison since models were trained and validated in similar conditions. Validation macro F1-score was chosen as the most relevant metric for the comparison of models due to unbalanced distribution of fire intensity classes. Accuracy and weighted F1-score were also taken into consideration.

SVM with an RBF kernel and C = 10 was the most successful model among all considered models. It had the highest validation macro F1-score, so it was the best one according to the main evaluation criterion used for the task. Neural Network was very close to the leading model in terms of its performance. It had a validation macro F1-score of 0.378. The worst performer according to validation macro F1-score was the Decision Tree model. However, the Decision Tree had the highest validation accuracy and the smallest training-validation gap in terms of macro F1-score. Thus, the difference in accuracy was not sufficient for evaluating models. Although the Decision Tree had the highest accuracy score, the validation macro F1-score was not the largest one. Hence, it was essential to consider macro F1-score as a measure because the provided data set was imbalanced. These differences can also be related to models' ways of separating data sets into classes. The Decision Tree utilised split-based rules, which made the model simpler and less prone to overfitting compared to other approaches to building models. Nevertheless, the Decision Tree was unable to capture more complicated relationships between attributes. Therefore, SVM with an RBF kernel and Neural Network created a more complicated separation between classes. As far as stability was concerned, the Decision Tree model was more stable than other models due to the nature of its approach to separating classes. The training and validation scores of macros F1 were quite close to each other. Specifically, their difference amounted to about 0.017. Consequently, it can be concluded that the chosen tree was not significantly overfitted. However, its relatively low validation macro F1-score indicated that the decision tree might have been too simplistic. In comparison to other models, both the Neural Network and SVM with an RBF kernel had relatively high validation macro F1-score values. Nonetheless, these two models had large training-validation gaps. SVM with an RBF kernel had a macro F1 gap of 0.563, whereas it was 0.617

for Neural Network. Therefore, it can be stated that these two models were prone to overfitting to the training set and thus failed to generalise well. It should be noted that more complex models do not necessarily outperform other models by all criteria. As has been shown in the previous analysis, the Decision Tree had better training stability than other models, while SVM and Neural Network models performed better about macro F1-score criterion.

E. Final Model Selection, Justification Evaluation

The final model chosen was SVM using RBF kernel with C = 10. This was chosen because it yielded the highest validation macro F1-score out of the three families of models. The rationale for using macro F1-score as the primary metric for selecting models is since the classes of interest are not balanced. There are more instances of Moderate and High fires while there are fewer instances of Extreme fire. If the model chosen based on accuracy, then there is a possibility of selection bias toward performing better on the more dominant classes. The selected SVM achieved:

Metric	Value
Validation accuracy	0.434
Validation macro F1-score	0.386
Validation weighted F1-score	0.426

The SVM was preferred over the Decision Tree as it obtained a higher validation macro F1 score. The Decision Tree demonstrated better stability and could be interpreted, but it performed worse when all four fire intensity levels had the same significance. As the objective was not limited to the identification of the two majority classes, the SVM model was considered a better choice. More importantly, selection of the model was not made according to just one performance parameter. The selection criteria included the following aspects: validation macro F1 score, validation accuracy, weighted F1 score, behavior of confusion matrices, classification of each class, stability of the model, and appropriateness for the given task. The RBF kernel SVM was chosen even though it was not ideal for the task.

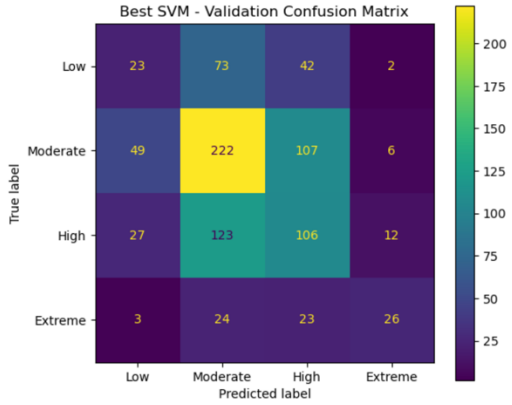

Figure E1. Best SVM confusion matrix

From the confusion matrix, we can see that the support vector machine continues to confuse a number of classes, particularly among Low, Moderate, and High classes. This implies that the class separators are not well-separated within the feature space. Furthermore, the model shows signs of overfitting since its training macro F1 score is significantly higher than its validation macro F1 score. Once we have chosen the final model, we proceed to train the model again using the entire labelled training dataset contained in wildfire_cls_train_full.csv. We do this so that the model will use all the information it can from the entire labelled training dataset before it makes predictions on the test dataset. The prediction file was saved as: 's4085541_predictions.csv'. The file contains one column, that is 'fire_intensity'. Finally, the selected Support Vector Machine model proved to be the best possible final model according to the used evaluation criteria. Nevertheless, the relatively low validation accuracy and the presence of overfitting make it clear that the selected model cannot be viewed as a perfect classifier despite its high quality.

F. Ethical considerations and Professional Responsibilities

As the given model deals with the problem of fire intensity prediction, there are important ethical and professional considerations to be considered about such a machine learning model. Namely, a model such as the one built might influence decisions about the prioritisation of fire events. Incorrect predictions can lead to serious negative consequences. The first type of error the model may make is underestimating the intensity of fires. Predicting an Extreme or High fire as Moderate or Low will result in an underestimation of the problem. This might lead to the underutilisation of resources, delayed warnings, and overall insufficient efforts in addressing the problem in a timely manner. In real life, underestimations such as these will likely endanger people's lives. The second type of error is overestimating the intensity of the fire. Predicting an Extreme fire where there is a Low or even Moderate fire event will cause unnecessary mobilisation of emergency response units. Such overestimations will likely cause resources to be wasted, making it impossible to respond to actual extreme situations at the right time. Both underestimating and overestimating the situation pose significant risks in emergencies. It becomes clear from the model validation process that the final model is not perfect. As indicated by the confusion matrix, SVM continues to confuse many of its classes. In this regard, it should not be considered as an adequate independent decision-making system. Rather, the model should be understood

as a decision-support system which provides valuable information but not as an alternative to expert judgment. The next important ethical consideration concerns the matter of informing the user about the reliability of different classes predictions. If the final model turns out to make better predictions for some groups of classes while performing inadequately for others, users should be informed about it. Thus, it can be said that the model may perform poorly when it comes to Low, Moderate, and High classes. An important aspect that should also be discussed is that of the dataset limitation. The given model was created based solely on the provided data. The assignment states that the training and testing data set includes fires located within seven regions and 35 countries, and therefore, the generalisation for other regions or countries should not be performed here. Thus, the model should be regarded as applicable only for the provided dataset. Lastly, another ethical aspect of using the given data set concerns the matter of restricting their application. According to the assignment specification, this dataset should only be used for the current assessment and should not be shared or further distributed.

G. Conclusion

This report proposed and evaluated three machine learning models for classification of wildfires intensity into four classes: Decision Tree, SVM and Neural Networks. This is a classical example of a supervised multi-classification problem, where the target variable is fire_intensity, and it has four classes: Low, Moderate, High and Extreme. The process included such steps as data inspection, exploratory data analysis, preprocessing, train/validation split, hyperparameter tuning experiments, model evaluation, comparison, model selection and test prediction generation. As shown by the exploratory analysis, there are several interesting challenges with these datasets, including missing values, different types of features, class imbalance and overlapping. The most interpretable and stable model was Decision Tree. Even though it has the smallest gap between training and validation metrics, it had a lower validation macro F1-score than both SVM and Neural Networks. On the contrary, Neural Networks showed the same results as SVMs; however, they were quite sensitive to learning rate and had the biggest gap between training and validation results. The best-performing model is an SVM with an RBF kernel and C = 10; its validation macro F1-score is the highest among the studied models. The last model, which was chosen based on the results obtained during experimentation, was refitted using the entire training labelled dataset and applied for prediction generation for the provided test dataset. The resulting file s4085541_predictions.csv with the only fire_intensity column was generated and saved. Even though the last and selected model performed the best among all other models under the specified evaluation framework, its validation performance can still be considered moderate. As seen from the confusion matrix, the model had troubles separating some of the fire intensities, especially those with similar categories. Thus, the final model is recommended to be treated as a tool that has some limitations. However, in a real-world situation of monitoring wildfire intensity, this model could still help making decisions while not being the only source of the information.

H. References

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). An introduction to statistical learning: With applications in Python. Springer. https://www.statlearning.com/

RMIT COSC2673 Course Materials. (2026a). Week 3 Lecture: Classification. RMIT University.

RMIT COSC2673 Course Materials. (2026b). Week 4 Lecture: Evaluation. RMIT University.

RMIT COSC2673 Course Materials. (2026c). Week 5 Lecture: Decision Trees. RMIT University.

RMIT COSC2673 Course Materials. (2026d). Week 6 Lecture: Support Vector Machines. RMIT University.

RMIT COSC2673 Course Materials. (2026e). Week 7 Lecture: Neural Networks. RMIT University.

RMIT COSC2673 Course Materials. (2026f). Week 8 Lecture: Deep Learning. RMIT University.

Scikit-learn developers. (n.d.). Decision trees. Scikit-learn. https://scikit-learn.org/stable/modules/tree.html

Scikit-learn developers. (n.d.). Support vector machines. Scikit-learn. https://scikit-learn.org/stable/modules/svm.html

Scikit-learn developers. (n.d.). Neural network models (supervised). Scikit-learn. https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Scikit-learn developers. (n.d.). Pipeline. Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html

Scikit-learn developers. (n.d.). ColumnTransformer. Scikit-learn. https://scikitlearn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html